



**UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO**

GABRIEL RODRIGUES DOS SANTOS

**AVALIAÇÃO DE DESEMPENHO NA TRANSFERÊNCIA DE ARQUIVOS COM AS
ARQUITETURAS CLIENTE-SERVIDOR E P2P: atividade 01 da unidade 2**

GABRIEL RODRIGUES DOS SANTOS

**AVALIAÇÃO DE DESEMPENHO NA TRANSFERÊNCIA DE ARQUIVOS COM AS
ARQUITETURAS CLIENTE-SERVIDOR E P2P: atividade 01 da unidade 2**

Relatório da Atividade 01 da Unidade 2
apresentado à disciplina Sistemas Distribuídos
(COMP0470) do Departamento de Computação
da Universidade Federal de Sergipe.

Professor: Prof. Dr. Rafael Oliveira
Vasconcelos.

SUMÁRIO

1	INTRODUÇÃO	3
2	TEMPO DE DISTRIBUIÇÃO DE UM ARQUIVO	3
3	IMPLEMENTAÇÃO	4
3.1	Cliente e servidores	4
3.2	Programa P2P	5
3.3	Limite de banda	7
4	EXPERIMENTOS	7
5	RESULTADOS	8
5.1	Tempos de transferência	8
5.2	Cliente-servidor	12
5.3	P2P	13
6	CONSIDERAÇÕES FINAIS	14
	REFERÊNCIAS	15

1 INTRODUÇÃO

Esta atividade compara o tempo necessário para distribuir um arquivo para vários clientes nas arquiteturas cliente-servidor e P2P. Na arquitetura cliente-servidor foram feitas três variações do servidor: uma que atende um cliente por vez, uma que atende todos os clientes ao mesmo tempo, com uma thread para cada cliente, e uma que atende no máximo um número fixo de clientes ao mesmo tempo, com um pool de threads. Para a arquitetura P2P foi implementado um programa baseado nas ideias do BitTorrent, em que o arquivo sai de uma única máquina e é dividido em pedaços que os próprios clientes repassam entre si.

Os experimentos variaram o tamanho do arquivo (5, 50 e 500 MB) e a quantidade de clientes (1, 2, 4 e 8). Para cada combinação são apresentados os tempos mínimo, médio e máximo de conclusão da transferência, que são comparados com os tempos teóricos vistos em aula. O código, os scripts dos experimentos e os resultados estão no repositório <https://github.com/Gabriell-Rodrigues/Atividade-01-Unidade-2-Sistema-Distribu-do->.

2 TEMPO DE DISTRIBUIÇÃO DE UM ARQUIVO

Na arquitetura cliente-servidor, o servidor precisa enviar uma cópia do arquivo para cada cliente. Sendo F o tamanho do arquivo, u_s a taxa de upload do servidor, N a quantidade de clientes e d_{min} a menor taxa de download entre eles, o tempo para distribuir o arquivo a todos os clientes é limitado pela Equação 1 (KUROSE; ROSS, 2021):

$$D_{cs} \geq \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{min}} \right\} \quad (1)$$

Com muitos clientes, o termo NF/u_s domina e o tempo cresce linearmente com N , porque o upload do servidor passa a ser o gargalo do sistema.

Na arquitetura P2P, cada par que já recebeu uma parte do arquivo pode enviá-la para os outros, então cada novo par também traz capacidade de upload. Sendo u_i a taxa de upload do par i , o tempo mínimo passa a ser o da Equação 2 (KUROSE; ROSS, 2021):

$$D_{P2P} \geq \max \left\{ \frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum_{i=1}^N u_i} \right\} \quad (2)$$

O primeiro termo indica que o servidor precisa enviar pelo menos uma cópia do arquivo, e o terceiro, que a quantidade total de dados (NF) é dividida pela capacidade de upload somada de todos os nós.

Nos experimentos deste trabalho todos os nós têm o upload limitado a 100 Mbit/s ($u_i = u_s$) e o download não é limitado, então o termo F/d_{min} pode ser desprezado. Com $u_i = u_s$, o terceiro termo da Equação 2 fica igual a $NF/((N+1)u_s)$, que é sempre menor que F/u_s . Assim, o tempo mínimo previsto para o P2P é F/u_s para qualquer N , enquanto no cliente-

servidor ele é N vezes esse valor. A Tabela 1 mostra F/u_s para os três arquivos, considerando $1 \text{ MB} = 2^{20} \text{ bytes}$ e $100 \text{ Mbit/s} = 10^8 \text{ bit/s}$.

Tabela 1 – Tempo para enviar uma cópia do arquivo a 100 Mbit/s

Arquivo	F (bytes)	F/u_s (s)
5 MB	5.242.880	0,42
50 MB	52.428.800	4,19
500 MB	524.288.000	41,94

Fonte: Elaborada pelo autor (2026).

No BitTorrent, que serviu de base para o programa P2P, o arquivo é dividido em pedaços (*chunks*) de 256 KB, e um servidor chamado rastreador (*tracker*) mantém a lista dos pares que participam da distribuição. Um par que chega obtém essa lista com o rastreador e passa a trocar pedaços diretamente com os outros pares, enviando pedaços enquanto ainda baixa os que faltam (COHEN, 2008; KUROSE; ROSS, 2021). Como a localização dos pares fica centralizada no rastreador e a transferência é feita entre os pares, Tanenbaum e Steen (2007) apresentam o BitTorrent como um exemplo de arquitetura híbrida.

3 IMPLEMENTAÇÃO

Os programas foram escritos em Java 17 e compilados com o Maven. A comunicação usa sockets TCP, com as classes `ServerSocket` e `Socket` (ORACLE, 2021b). O TCP foi escolhido por oferecer entrega confiável e comunicação por fluxo de bytes, o que atende à transferência de um arquivo inteiro. O Quadro 1 lista as classes do projeto.

Quadro 1 – Classes do projeto

Classe	Função
<code>GeradorArquivos</code>	gera os arquivos de teste de 5, 50 e 500 MB
<code>Transferencia</code>	envio e recebimento do arquivo no cliente-servidor
<code>ServidorSequencial</code>	servidor que atende um cliente por vez
<code>ServidorThreads</code>	servidor com uma thread para cada cliente
<code>ServidorPool</code>	servidor com pool de threads
<code>Cliente</code>	conecta ao servidor, recebe o arquivo e mede o tempo
<code>Tracker</code>	rastreador do programa P2P
<code>Peer</code>	par do programa P2P, que baixa e envia pedaços
<code>LimiteBanda</code>	limita a taxa de upload de cada processo

Fonte: Elaborado pelo autor (2026).

3.1 Cliente e servidores

Ao receber uma conexão, o servidor envia o tamanho do arquivo (8 bytes) e depois o conteúdo, em blocos de 64 KB. O cliente lê o tamanho, recebe os bytes até completar esse total

e os descarta, como permitido no enunciado, sem gravar o arquivo em disco. O tempo é medido pelo próprio cliente, do momento em que ele pede a conexão até a chegada do último byte, de modo que inclui o tempo de espera na fila do servidor.

As três variações do servidor diferem apenas no que é feito com a conexão devolvida pelo accept, como mostra a Listagem 1. O servidor sequencial envia o arquivo na mesma thread que aceita as conexões, então os outros clientes ficam esperando na fila de conexões até o envio atual terminar. É o que Tanenbaum e Steen (2007) chamam de servidor iterativo. Os outros dois são servidores concorrentes. O segundo cria uma thread para cada conexão aceita, como na arquitetura de uma thread por conexão (*thread-per-connection*) descrita por Coulouris et al. (2013). O terceiro entrega a conexão a um pool com número fixo de threads, a arquitetura de pool de trabalhadores (*worker pool*). O pool foi criado com `Executors.newFixedThreadPool`, em que as tarefas que chegam quando todas as threads estão ocupadas esperam em uma fila até uma delas ficar livre (ORACLE, 2021a). Assim, o servidor atende ao mesmo tempo no máximo tantos clientes quanto o número de threads do pool, e os demais esperam a sua vez.

Listagem 1 – Tratamento de cada conexão nos três servidores

```
// ServidorSequencial
Socket cliente = servidor.accept();
Transferencia.atender(cliente, arquivo);

// ServidorThreads
Socket cliente = servidor.accept();
new Thread(() -> Transferencia.atender(cliente, arquivo)).start();

// ServidorPool
ExecutorService pool = Executors.newFixedThreadPool(maximo);
...
Socket cliente = servidor.accept();
pool.execute(() -> Transferencia.atender(cliente, arquivo));
```

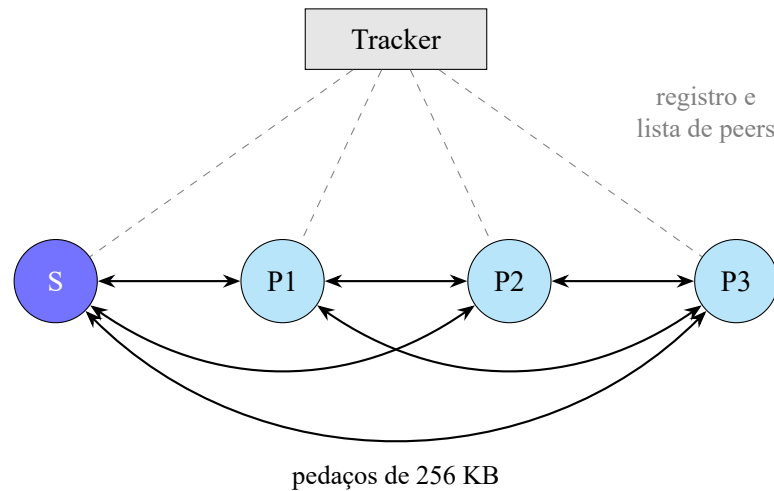
Fonte: Elaborada pelo autor (2026).

3.2 Programa P2P

O programa P2P tem dois componentes, o Tracker e o Peer. Ao iniciar, cada peer (par) se registra no tracker informando a porta em que aceita conexões e recebe o tamanho do arquivo e a lista dos outros peers. O semeador é o peer que começa com o arquivo completo e é a única máquina que tem o arquivo no início; os demais peers começam sem nenhum pedaço. Como no BitTorrent, o arquivo é dividido em pedaços de 256 KB, o tamanho mais comum segundo a especificação do protocolo (COHEN, 2008), o que dá 20 pedaços no arquivo de 5 MB, 200 no de 50 MB e 2.000 no de 500 MB. A Figura 1 mostra os componentes com três peers.

Para cada outro peer da lista, o peer abre uma conexão TCP e cria uma thread que baixa pedaços por ela, usando as mensagens do Quadro 2. A thread pede primeiro o mapa de pedaços do outro peer, que indica quais pedaços ele já tem, e depois pede, um de cada vez, pedaços escolhidos ao acaso entre os que o outro tem, que ainda faltam e que não estão sendo baixados

Figura 1 – Componentes do programa P2P



Fonte: Elaborada pelo autor (2026).

Legenda: S = semeador; P = peer que baixa o arquivo.

por outra thread. Quando o outro peer não tem nenhum pedaço útil, a thread espera 100 ms e pede o mapa de novo. Cada pedaço recebido é gravado na sua posição em um arquivo temporário.

Ao mesmo tempo, outra thread aceita as conexões dos outros peers e cria uma thread para atender cada uma, enviando os pedaços já recebidos. Dessa forma o peer envia pedaços enquanto ainda está baixando o arquivo, que é a ideia principal do BitTorrent. A lista de peers é consultada no tracker a cada segundo, para que um peer que entrou depois também seja usado. Quando tem todos os pedaços, o peer mostra o tempo e de quais peers recebeu os pedaços, e continua compartilhando o arquivo.

Quadro 2 – Mensagens do programa P2P

Mensagem	De / para	Resposta
registro (porta e tamanho do arquivo, ou -1 se não tem o arquivo)	peer / tracker	tamanho do arquivo e lista dos outros peers (IP e porta)
M (pedido do mapa)	peer / peer	mapa de bits com os pedaços que o peer tem
P e índice do pedaço	peer / peer	tamanho e conteúdo do pedaço, ou -1 se o peer não tem o pedaço

Fonte: Elaborado pelo autor (2026).

Os três aspectos do BitTorrent destacados em aula (com quais pares se conectar, quais partes pedir e para quais pares enviar) foram resolvidos de forma simples, porque os experimentos têm no máximo nove peers: cada peer se conecta com todos os outros, pede os pedaços em ordem aleatória e envia pedaços para qualquer peer que pedir. A ordem aleatória é a indicada na especificação do protocolo, e ajuda a evitar que os pedaços de um peer sejam apenas um subconjunto dos pedaços de outro (COHEN, 2008). Não foram implementadas a política de pedir

primeiro os pedaços mais raros nem o mecanismo olho por olho (*tit-for-tat*), que o BitTorrent usa para escolher para quais pares enviar (KUROSE; ROSS, 2021).

Para conferir a implementação, o programa foi executado com o arquivo de 50 MB e três peers, e o resumo SHA-256 do arquivo montado por cada peer foi igual ao do arquivo original.

3.3 Limite de banda

Os experimentos foram executados em uma única máquina, com cada nó em um container Docker. Sem limite, a transferência entre containers da mesma máquina é feita pela memória e o tempo dependeria da CPU e do disco, e não da taxa de upload considerada nas equações da seção 2. Por isso, o upload de cada nó foi limitado a 100 Mbit/s.

A primeira opção seria limitar a interface de rede de cada container com o comando `tc` do Linux, mas o núcleo do WSL 2, onde o Docker foi executado, não tem as disciplinas de fila usadas por ele (`tbfb`, `htb` e `netem`). O limite foi então feito no próprio programa, na classe `LimiteBanda`: antes de escrever cada bloco de até 64 KB no socket, a thread reserva o próximo intervalo de tempo livre, calculado a partir da taxa configurada, e espera até ele começar. A reserva é compartilhada por todas as threads do processo, como acontece com a placa de rede de uma máquina, então, quando o servidor atende oito clientes ao mesmo tempo, cada um recebe em média um oitavo da taxa. Como a thread pode acordar um pouco depois do previsto, até 50 ms de atraso podem ser compensados nas reservas seguintes, para que a taxa média fique igual à configurada. O limite vale para o upload do servidor e de todos os peers, inclusive o semeador, e é definido pela variável de ambiente `BANDA_MBPS`. O download não é limitado.

4 EXPERIMENTOS

Os experimentos foram executados em um notebook com processador Intel Core i3-1215U, 12 GB de memória e SSD NVMe, com Windows 11. O Docker Engine 29.1.3 e o Docker Compose 2.40.3 foram executados no Ubuntu 24.04 do WSL 2, que tinha 8 processadores lógicos e 5,8 GB de memória disponíveis. Dentro dos containers foi usado o Java 17 (Temurin 17.0.20).

Cada nó roda em um container separado, todos criados a partir da mesma imagem e ligados à rede padrão do Docker Compose, na qual um container encontra o outro pelo nome do serviço (DOCKER, 2026b). O `docker-compose.yml` define os serviços do Quadro 3, e a quantidade de clientes ou de peers é escolhida com a opção `--scale` do comando `docker compose up` (DOCKER, 2026a).

O script `scripts/experimentos.sh` executa todas as combinações de arquitetura (servidor sequencial, servidor com uma thread por cliente, servidor com pool de 2 threads e P2P), tamanho do arquivo (5, 50 e 500 MB) e quantidade de clientes (1, 2, 4 e 8), com três repetições, o que dá 144 execuções. O pool foi configurado com 2 threads para que, com 4 e 8 clientes, parte deles tenha de esperar na fila. Antes de cada execução todos os containers são removidos,

Quadro 3 – Serviços do `docker-compose.yml`

Serviço	O que executa
gerador	GeradorArquivos, gravando os arquivos em um volume compartilhado
servidor	um dos três servidores, escolhido pela variável <code>MOD0</code>
cliente	Cliente, com uma réplica para cada cliente
tracker	Tracker
semeador	Peer com o arquivo completo
peer	Peer sem o arquivo, com uma réplica para cada cliente

Fonte: Elaborado pelo autor (2026).

e o servidor, ou o tracker e o semeador no caso do P2P, é iniciado primeiro. O script define então um instante de início alguns segundos à frente, e todos os clientes esperam esse instante para começar, para que os pedidos cheguem ao mesmo tempo. No P2P o tempo é medido pelo peer desde o momento em que ele obtém o tamanho do arquivo com o tracker, logo após esse instante, até ter todos os pedaços, e o semeador não é contado como cliente: com $N = 8$ há oito peers baixando o arquivo, além do semeador.

O tempo de cada cliente é gravado em `resultados/tempos.csv` e, no P2P, a quantidade de pedaços que cada peer recebeu do semeador vai para `resultados/origem_p2p.csv`. Depois, o script `scripts/analisar.py` calcula os tempos mínimo, médio e máximo e gera os gráficos. Em cada combinação, esses valores foram calculados sobre os tempos de todos os clientes das três repetições, ou seja, sobre $3N$ valores. O tempo máximo é o tempo para que todos os clientes recebam o arquivo, que corresponde a D_{cs} e D_{P2P} nas equações da seção 2. A Listagem 2 mostra os comandos usados. A campanha completa durou cerca de três horas, sem nenhuma falha.

Listagem 2 – Comandos para executar os experimentos

<code>bash scripts/experimentos.sh</code>	# gera os arquivos e executa as 144 combinacoes
<code>python scripts/analisar.py</code>	# calcula minimo, medio e maximo e gera os graficos

Fonte: Elaborada pelo autor (2026).

5 RESULTADOS

5.1 Tempos de transferência

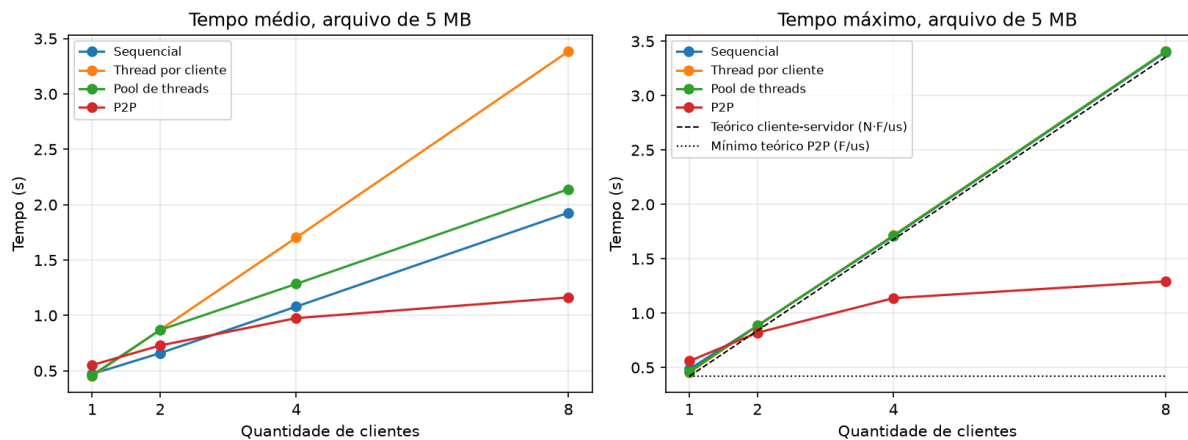
As Tabelas 2, 3 e 4 mostram os tempos mínimo, médio e máximo de cada combinação, e as Figuras 2, 3 e 4 mostram os tempos médio e máximo em função da quantidade de clientes. Nos gráficos do tempo máximo também estão as curvas teóricas NF/u_s , do cliente-servidor, e F/u_s , o mínimo do P2P.

Tabela 2 – Tempos de transferência do arquivo de 5 MB, em segundos

Clientes	Arquitetura	Mínimo	Médio	Máximo
1	Sequencial	0,46	0,47	0,48
	Thread por cliente	0,45	0,45	0,45
	Pool de 2 threads	0,46	0,46	0,46
	P2P	0,55	0,55	0,56
2	Sequencial	0,44	0,66	0,88
	Thread por cliente	0,86	0,87	0,88
	Pool de 2 threads	0,86	0,87	0,88
	P2P	0,67	0,73	0,82
4	Sequencial	0,45	1,08	1,71
	Thread por cliente	1,69	1,70	1,72
	Pool de 2 threads	0,86	1,28	1,71
	P2P	0,71	0,98	1,14
8	Sequencial	0,46	1,93	3,40
	Thread por cliente	3,36	3,38	3,41
	Pool de 2 threads	0,88	2,14	3,41
	P2P	1,02	1,16	1,29

Fonte: Elaborada pelo autor (2026).

Figura 2 – Tempos médio e máximo com o arquivo de 5 MB



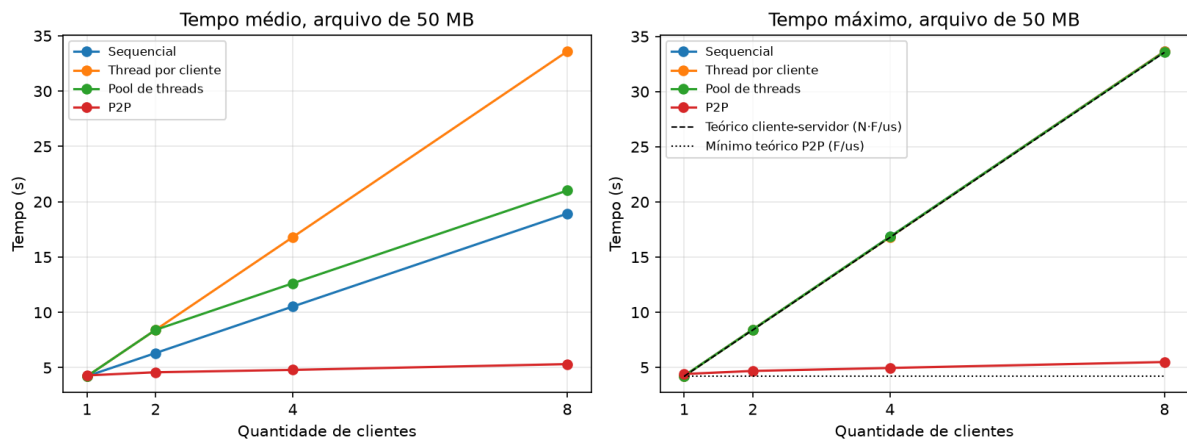
Fonte: Elaborada pelo autor (2026).

Tabela 3 – Tempos de transferência do arquivo de 50 MB, em segundos

Cientes	Arquitetura	Mínimo	Médio	Máximo
1	Sequencial	4,22	4,23	4,24
	Thread por cliente	4,22	4,22	4,23
	Pool de 2 threads	4,22	4,22	4,23
	P2P	4,24	4,30	4,40
2	Sequencial	4,22	6,32	8,43
	Thread por cliente	8,41	8,42	8,43
	Pool de 2 threads	8,41	8,42	8,43
	P2P	4,51	4,59	4,68
4	Sequencial	4,22	10,53	16,84
	Thread por cliente	16,78	16,80	16,81
	Pool de 2 threads	8,42	12,63	16,84
	P2P	4,67	4,80	4,95
8	Sequencial	4,23	18,93	33,64
	Thread por cliente	33,52	33,59	33,64
	Pool de 2 threads	8,42	21,01	33,61
	P2P	5,19	5,32	5,49

Fonte: Elaborada pelo autor (2026).

Figura 3 – Tempos médio e máximo com o arquivo de 50 MB



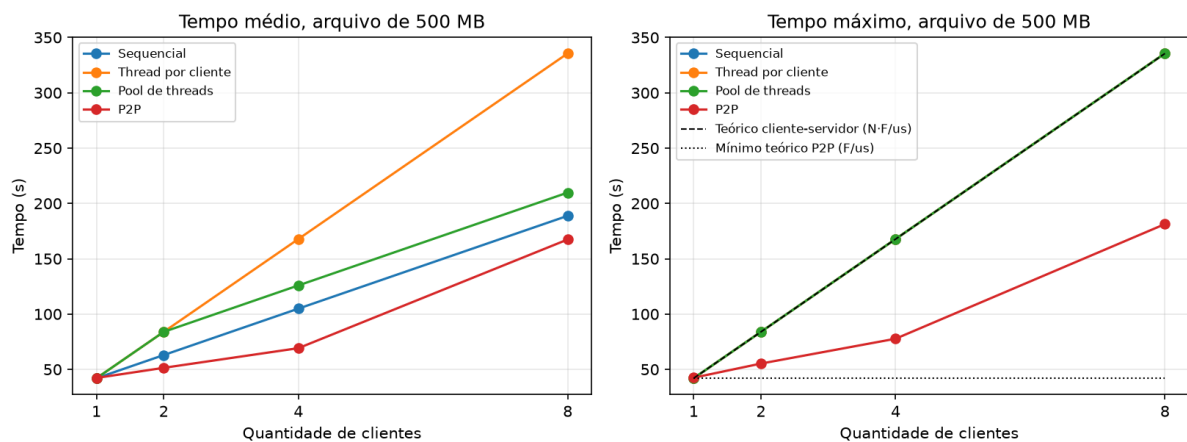
Fonte: Elaborada pelo autor (2026).

Tabela 4 – Tempos de transferência do arquivo de 500 MB, em segundos

Clientes	Arquitetura	Mínimo	Médio	Máximo
1	Sequencial	41,97	41,97	41,98
	Thread por cliente	41,97	41,97	41,97
	Pool de 2 threads	41,97	41,97	41,98
	P2P	42,11	42,24	42,45
2	Sequencial	41,96	62,94	83,92
	Thread por cliente	83,90	83,91	83,92
	Pool de 2 threads	83,89	83,91	83,92
	P2P	47,17	51,41	55,23
4	Sequencial	41,98	104,91	167,84
	Thread por cliente	167,82	167,82	167,83
	Pool de 2 threads	83,94	125,89	167,85
	P2P	53,24	69,18	77,70
8	Sequencial	42,00	188,80	335,62
	Thread por cliente	335,56	335,59	335,63
	Pool de 2 threads	83,95	209,78	335,62
	P2P	151,87	167,41	181,31

Fonte: Elaborada pelo autor (2026).

Figura 4 – Tempos médio e máximo com o arquivo de 500 MB



Fonte: Elaborada pelo autor (2026).

5.2 Cliente-servidor

Com um cliente, as três variações levaram o mesmo tempo, muito próximo de F/u_s (4,22 s contra 4,19 s no arquivo de 50 MB e 41,97 s contra 41,94 s no de 500 MB), o que mostra que o limite de banda funcionou como esperado. Com mais clientes, o tempo máximo das três variações foi praticamente o mesmo e cresceu linearmente com N , acompanhando a curva NF/u_s nos gráficos. Isso acontece porque, nas três, o servidor precisa enviar N cópias pelo mesmo upload de 100 Mbit/s, e a forma de atender os clientes não muda a quantidade de dados enviada.

O que muda entre as variações é a distribuição dos tempos entre os clientes. Sendo $T = F/u_s$ o tempo para enviar uma cópia:

- no servidor sequencial, o k -ésimo cliente atendido termina em kT , então o mínimo é T , o médio é $(N + 1)T/2$ e o máximo é NT ;
- com uma thread por cliente, os N envios dividem o upload e todos terminam juntos, em NT , de modo que o mínimo, o médio e o máximo são praticamente iguais;
- com o pool de 2 threads, os clientes são atendidos de dois em dois, e os pares terminam em $2T, 4T$, até NT , com médio igual a $(N/2 + 1)T$.

A Tabela 5 compara os tempos médio e máximo previstos dessa forma com os medidos no arquivo de 500 MB. A diferença ficou abaixo de 0,1% em todos os casos.

Tabela 5 – Tempos previstos e medidos no cliente-servidor com o arquivo de 500 MB, em segundos

Arquitetura	Clientes	Tempo médio		Tempo máximo	
		Previsto	Medido	Previsto	Medido
Sequencial	2	62,91	62,94	83,89	83,92
	4	104,86	104,91	167,77	167,84
	8	188,74	188,80	335,54	335,62
Thread por cliente	2	83,89	83,91	83,89	83,92
	4	167,77	167,82	167,77	167,83
	8	335,54	335,59	335,54	335,63
Pool de 2 threads	2	83,89	83,91	83,89	83,92
	4	125,83	125,89	167,77	167,85
	8	209,72	209,78	335,54	335,62

Fonte: Elaborada pelo autor (2026).

Com isso, o servidor sequencial teve o menor tempo médio entre as três variações e o servidor com uma thread por cliente, o maior: dividir o upload entre todos atrasa todos os clientes sem fazer o último terminar antes. Atender vários clientes ao mesmo tempo traz vantagem quando o gargalo não é o upload do servidor, por exemplo quando os clientes têm download lento. Neste experimento, em que todos pedem o mesmo arquivo e o upload do servidor é o gargalo, essa vantagem não aparece.

5.3 P2P

No P2P o tempo cresceu muito menos com a quantidade de clientes. Com 8 clientes, o tempo máximo foi de 1,29 s contra 3,41 s do cliente-servidor no arquivo de 5 MB, de 5,49 s contra 33,64 s no de 50 MB e de 181,31 s contra 335,63 s no de 500 MB. Isso acontece porque cada peer que recebe um pedaço passa a enviá-lo para os outros, como considerado na Equação 2, e o semeador não precisa enviar o arquivo inteiro para cada um. Com 500 MB, cada peer recebeu do semeador 50,5% dos pedaços quando havia 2 peers, 25,5% com 4 e 13,2% com 8, e o restante veio dos outros peers (Tabela 6). Com um único peer, o P2P foi um pouco mais lento que o cliente-servidor (4,30 s contra cerca de 4,22 s de média com 50 MB), porque cada pedaço de 256 KB é pedido separadamente, enquanto o servidor envia o arquivo em um fluxo contínuo.

Nos arquivos de 5 e 50 MB os tempos ficaram acima do mínimo F/u_s , e a diferença é explicada pela quantidade de dados enviada pelo semeador. A Equação 2 supõe que o semeador envia cada pedaço uma única vez. No programa, porém, cada peer escolhe os pedaços ao acaso sem saber o que os outros estão pedindo, e no começo da transferência todos pedem pedaços ao semeador ao mesmo tempo, então um mesmo pedaço pode ser enviado pelo semeador a mais de um peer. Multiplicando a porcentagem de pedaços recebidos do semeador pela quantidade de peers obtém-se quantas cópias do arquivo o semeador enviou no total. Como o upload do semeador é o gargalo, o tempo previsto passa a ser esse número de cópias vezes F/u_s . Na Tabela 6, esse valor ficou próximo do tempo médio medido em todos os casos com 5 e 50 MB, com diferença menor que 5%.

Tabela 6 – Pedaços enviados pelo semeador e tempos do P2P

Arquivo (MB)	Peers	Do semeador (%)	Cópias enviadas pelo semeador	Previsto (s)	Médio (s)	Máximo (s)
5	2	86,7	1,73	0,73	0,73	0,82
	4	60,8	2,43	1,02	0,98	1,14
	8	33,1	2,65	1,11	1,16	1,29
50	2	54,2	1,08	4,55	4,59	4,68
	4	28,4	1,14	4,77	4,80	4,95
	8	15,7	1,25	5,26	5,32	5,49
500	2	50,5	1,01	42,36	51,41	55,23
	4	25,5	1,02	42,82	69,18	77,70
	8	13,2	1,06	44,28	167,41	181,31

Fonte: Elaborada pelo autor (2026).

Nota: *Do semeador* é a média, por peer, da porcentagem de pedaços recebidos do semeador; *Previsto* é o número de cópias enviadas pelo semeador multiplicado por F/u_s .

As cópias repetidas são mais frequentes no arquivo de 5 MB, que tem apenas 20 pedaços: com 8 peers, o semeador enviou o equivalente a 2,65 cópias do arquivo. Uma escolha que evitasse pedir ao semeador pedaços que outro peer já tem reduziria essas cópias, que é a ideia da política do BitTorrent de pedir primeiro os pedaços mais raros (KUROSE; ROSS, 2021).

Com 500 MB o semeador enviou pouco mais de uma cópia, mas os tempos com 2 ou mais peers ficaram acima do previsto, e a diferença cresceu com a quantidade de peers. Nesse caso o limite não foi a rede. Diferente do cliente, que descarta os dados, o peer precisa guardar os pedaços para enviá-los aos outros, e como 500 MB não cabem na memória reservada para cada processo Java (256 MB), os pedaços são gravados em um arquivo no disco. Com 8 peers são 4 GB gravados no mesmo disco virtual, quase toda a memória do WSL (5,8 GB).

Para verificar isso, as execuções de 500 MB com 4 e 8 peers foram repetidas uma vez, acompanhando o sistema com o `vmstat` a cada 2 segundos. Os tempos ficaram próximos aos da campanha (76,0 s com 4 peers e 154,9 s com 8). Na execução com 8 peers, em 69 das 79 amostras havia três ou mais processos bloqueados esperando o disco, chegando a 25, e a CPU passou em média 66% do tempo em espera de E/S. Houve trechos de até 20 s com processos bloqueados e sem nenhuma leitura ou escrita concluída, alternados com momentos de escrita intensa no disco, de até cerca de 175 MB/s. Nas duas execuções a vazão total recebida pelos peers ficou perto de 26 MB/s (2.000 MB em 76 s e 4.000 MB em 155 s), mesmo com o dobro de dados no segundo caso, enquanto a soma dos uploads dos nós, com 8 peers, era de 900 Mbit/s (cerca de 107 MB/s). Isso indica que, com 500 MB, o gargalo passou a ser o disco da máquina compartilhado por todos os containers, um recurso que a Equação 2 não considera e que, em uma rede real, seria separado em cada máquina. Também por isso os tempos de 500 MB variaram mais entre as repetições (de 151,87 s a 181,31 s com 8 peers). Mesmo assim, com 8 clientes o P2P levou cerca de metade do tempo do cliente-servidor.

6 CONSIDERAÇÕES FINAIS

Os experimentos reproduziram o comportamento visto em aula. No cliente-servidor, o tempo para todos os clientes receberem o arquivo cresceu linearmente com a quantidade de clientes nas três variações do servidor, igual a NF/u_s , porque o upload do servidor é o gargalo. A variação escolhida muda apenas a ordem em que os clientes terminam, e o servidor sequencial teve o menor tempo médio. No P2P, os próprios clientes repassaram a maior parte dos pedaços e o tempo cresceu pouco com a quantidade de clientes: com 8 clientes e o arquivo de 50 MB, a transferência levou no máximo 5,49 s, contra 33,64 s no cliente-servidor. As diferenças em relação ao mínimo teórico foram explicadas pelas cópias repetidas enviadas pelo semeador e, no arquivo de 500 MB, pelo disco compartilhado entre os containers da mesma máquina.

O código, os scripts e os resultados (tempos de cada cliente, resumo e gráficos) estão no repositório do projeto, com as instruções de execução no arquivo `README.md`.

REFERÊNCIAS

COHEN, Bram. **The BitTorrent protocol specification**. [S. l.], 2008. Disponível em: https://www.bittorrent.org/beps/bep_0003.html. Acesso em: 2 out. 2026.

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. **Sistemas distribuídos: conceitos e projeto**. 5. ed. Porto Alegre: Bookman, 2013.

DOCKER. **docker compose up**. [S. l.], 2026a. Disponível em: <https://docs.docker.com/reference/cli/docker/compose/up/>. Acesso em: 2 out. 2026.

DOCKER. **Networking in Compose**. [S. l.], 2026b. Disponível em: <https://docs.docker.com/compose/how-tos/networking/>. Acesso em: 2 out. 2026.

KUROSE, James F.; ROSS, Keith W. **Redes de computadores e a Internet: uma abordagem top-down**. 8. ed. Porto Alegre: Bookman, 2021.

ORACLE. **Executors (Java SE 17 & JDK 17)**. [S. l.], 2021a. Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/Executors.html>. Acesso em: 2 out. 2026.

ORACLE. **ServerSocket (Java SE 17 & JDK 17)**. [S. l.], 2021b. Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/ServerSocket.html>. Acesso em: 2 out. 2026.

TANENBAUM, Andrew S.; STEEN, Maarten van. **Sistemas distribuídos: princípios e paradigmas**. 2. ed. São Paulo: Pearson Prentice Hall, 2007.